Introduction to Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP) is a method of writing programs using **objects** and **classes**. It helps make code more organized, reusable, and easier to understand.

Why use OOP?

- Organize code into **real-world entities**
- Make code **modular and reusable**
- Improve **data security and structure**
- Reduce code duplication and improve maintainability

Key Concepts in Python OOP

1. **Class**: A blueprint to create objects. It defines properties (variables) and methods (functions).
2. **Object**: An instance of a class. It represents a real-world entity.
3. **Encapsulation**: Hiding internal details and only exposing what is necessary.
4. **Abstraction**: Showing only essential features and hiding complex details.
5. **Inheritance**: One class can use properties and methods of another class.
6. **Polymorphism**: Same function name behaves differently for different classes.

Simple Example

```
class Car:
    name="Neeraj"
    game="Card Game"
    def start(self):
        print("Car started")

my_car = Car()
print(my_car.name)
my_car.start()
```

In the example above:

- `Car` is a **class**
- `my_car` is an **object**
- `start()` is a **method**

What is a Constructor?

- It is a function that runs **automatically** when an object is created.
- In Python, the constructor method is named `__init__()`.
- It is used to assign **initial values** to object properties (like `name`, `age`, etc.).

Syntax of Constructor in Python

```
class ClassName:
```

```
    def __init__(self, parameters):
        # code to initialize the object
```

Example of Constructor

```
class Student:
    def __init__(self, name, marks):   # constructor
        self.name = name
        self.marks = marks

    def show(self):
        print("Name:", self.name)
        print("Marks:", self.marks)

s1 = Student("Neeraj", 90)   # object created, constructor
runs
s1.show()
```

Output:
Name: Neeraj
Marks: 90

Types of Constructors in Python

Type	Description
Default Constructor	Constructor with no parameters
Parameterized Constructor	Constructor with parameters to initialize values

Default Constructor Example:

```
class Demo:
    def __init__(self):      # No parameters
        print("Object Created")

obj = Demo()
```

Parameterized Constructor Example:

```
class Demo:
    def __init__(self, message):   # With parameter
        print("Message:", message)

obj = Demo("Hello Python")
```

Summary

- `__init__()` is a **constructor**
- It runs **automatically** when an object is created
- Used to **initialize** object variables

What is a Decorator?

A **decorator** in Python is a function that **adds extra functionality** to another function **without changing its structure**.

Think of it like "wrapping" a function inside another function to **extend its behavior**.

Why Use Decorators?

- Code **reuse**
- Add **logging**, **authentication**, **timing**, etc.
- Cleaner and more Pythonic way to modify functions

Basic Structure of a Decorator

```
def decorator_function(original_function):
    def wrapper_function():
        print("Before the function runs")
        original_function()
        print("After the function runs")
    return wrapper_function
```

Example Without @ Syntax:

```
def greet():
    print("Hello!")

def decorator(func):
    def wrapper():
        print("Welcome")
        func()
        print("Goodbye")
    return wrapper

decorated_greet = decorator(greet)
decorated_greet()
```

Output:

```
Welcome
Hello!
Goodbye
```

Example With @decorator Syntax (Shortcut):

```
def decorator(func):
    def wrapper():
        print("Welcome")
        func()
        print("Goodbye")
    return wrapper
```

```
@decorator
def greet():
    print("Hello!")

greet()
```

Decorator with Arguments

```
def decorator(func):
    def wrapper(name):
        print("Welcome")
        func(name)
        print("Goodbye")
    return wrapper

@decorator
def greet(name):
    print("Hello", name)

greet("Neeraj")
```

Example Using a Function with Arguments

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Arguments passed:", args, kwargs)
        result = func(*args, **kwargs)
        print("Function executed successfully.")
        return result
    return wrapper

@my_decorator
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

greet("Neeraj")
greet("Anjali", greeting="Hi")
```

output:

Arguments passed: ('Neeraj',) {}
Hello, Neeraj!
Function executed successfully.

Arguments passed: ('Anjali',) {'greeting': 'Hi'}
Hi, Anjali!
Function executed successfully.

Summary

Term	Meaning
`@decorator`	Shortcut to apply a wrapper to a function
`wrapper()`	Inner function that modifies the behavior
Useful For	Logging, timing, access control, etc.

Getters and Setters in Python

Getters are methods used to **get/access** the value of a private attribute.
Setters are methods used to **set/update** the value of a private attribute.
They help in **encapsulation**, which is one of the core concepts of **Object-Oriented Programming (OOP)**.

Why Use Getters and Setters?

- To **protect** direct access to private attributes.
- To **validate** data before setting it.
- To **control** how an attribute is accessed or modified.
- To implement **read-only** or **write-only** properties.

Syntax (Using Methods)

```
class Student:
    def __init__(self):
        self.__name = ""

    # Getter method
    def get_name(self):
        return self.__name

    # Setter method
    def set_name(self, name):
        if len(name) > 0:
            self.__name = name
        else:
            print("Name cannot be empty")

# Usage
s = Student()
s.set_name("Alice")
print(s.get_name())
```

Using `@property` Decorator (Pythonic Way)

```
class Student:
    def __init__(self):
        self.__name = ""

    @property
    def name(self):           # Getter
        return self.__name

    @name.setter
    def name(self, value):    # Setter
```

```
        if len(value) > 0:
            self.__name = value
        else:
            print("Invalid name")

# Usage
s = Student()
s.name = "Bob"       # Calls setter
print(s.name)        # Calls getter
```

Key Points

Feature	Explanation
`__var`	Double underscore makes variable private
`@property`	Turns method into a getter
`@varname.setter`	Defines setter for the same property
Encapsulation	Prevents direct access to attributes
Validation	Can be added inside setter

Inheritance in python

Code **reusability**: You don't have to write the same code again.
It helps in **organizing** code in a hierarchical manner.
You can **extend** or **modify** behavior of the parent class in the child class.

Basic Syntax of Inheritance:

```
class Parent:
    def display(self):
        print("This is Parent class.")

class Child(Parent):  # Inheriting Parent class
    def show(self):
        print("This is Child class.")

obj = Child()
obj.display()  # Accessing Parent class method
obj.show()     # Accessing Child class method
```

Types of Inheritance in Python:

Type	Description	Example
Single	One child class inherits from one parent class	`class B(A)`
Multiple	One child inherits from more than one parent class	`class C(A, B)`
Multilevel	One class inherits from a child class which in turn inherits from another	`class C(B), class B(A)`
Hierarchical	Multiple child classes inherit from the same parent class	`class B(A), class C(A)`
Hybrid	Combination of more than one type of inheritance	Combination of above

Access Modifiers in Python

In Python, **access modifiers** are used to **control the visibility (accessibility)** of class members (variables and methods). Unlike some other languages like Java or C++, Python doesn't have strict access control, but it provides **naming conventions** to simulate them.

Types of Access Modifiers in Python

Modifier	Syntax Example	Access Level
Public	`self.name`	Accessible from anywhere
Protected	`self._name`	Suggests access within class and subclass
Private	`self.__name`	Not accessible outside the class (name mangled)

---

1. Public Members

Can be accessed **anywhere** (inside or outside the class).

```
class Student:
  def __init__(self):
    self.name = "Neeraj" # Public member

s = Student()
print(s.name) # Accessible
```

---

2. Protected Members

Defined with a **single underscore _**.
By convention, should not be accessed outside the class or its subclasses.
Python doesn't enforce this restriction.

```
class Student:
  def __init__(self):
    self._marks = 90 # Protected
```

```
class Derived(Student):
  def show(self):
    print("Marks:", self._marks)

d = Derived()
d.show()
print(d._marks) # Technically accessible, but not recommended
```

---

3. Private Members

Defined with **double underscore __**.
Python uses **name mangling** to make them harder to access from outside.

```
class Student:
  def __init__(self):
    self.__roll = 101 # Private

  def show(self):
    print("Roll No:", self.__roll)

s = Student()
s.show()
# print(s.__roll)  # Error: 'Student' object has no attribute
'__roll'
print(s._Student__roll) # Accessible via name mangling (not recommended)
```

Summary Table:

Modifier	Prefix	Access From Outside	Subclass Access	Notes
Public	none	Yes	Yes	Default access
Protected	`_var`	Yes (by convention no)	Yes	Just a naming convention
Private	`__var`	No (name mangled)	No (directly)	Still accessible via workaround

Tip:

Python focuses more on **developer discipline** than strict enforcement. Use private/protected when needed, but don't rely on them for security.